

Customers

cust_id	cust_name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
6	Frank
7	Grace
8	Hannah
9	Ivy
10	Jack

Order

order_id	cust_id	order_date	amount
101	1	2023-01-05	1200.00
102	2	2023-01-10	800.00
103	1	2023-02-15	600.00
104	3	2023-02-20	1500.00
105	4	2023-01-25	300.00
106	2	2023-02-05	2200.00
107	5	2023-03-12	450.00
108	6	2023-03-15	5200.00
109	1	2023-03-20	700.00
110	7	2023-01-18	950.00
111	8	2023-02-10	120.00
112	9	2023-01-22	640.00

- 1.Find customers who placed at least one order
- 2.Count total orders per customer
- 3.Identify high-value orders("High" if amount > 1000, "Medium" if between 500 and 1000, "Low" otherwise)
- 4.Find customers with no orders
- 5.Find latest order per customer

Customers

cust_id	cust_name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
6	Frank
7	Grace
8	Hannah
9	Ivy
10	Jack

Order

order_id	cust_id	order_date	amount
101	1	2023-01-05	1200.00
102	2	2023-01-10	800.00
103	1	2023-02-15	600.00
104	3	2023-02-20	1500.00
105	4	2023-01-25	300.00
106	2	2023-02-05	2200.00
107	5	2023-03-12	450.00
108	6	2023-03-15	5200.00
109	1	2023-03-20	700.00
110	7	2023-01-18	950.00
111	8	2023-02-10	120.00
112	9	2023-01-22	640.00

- 1.Total revenue by customer
- 2.Categorize customers based on revenue("Gold" if revenue > 5000, "Silver" if between 2000 and 5000,"Bronze" otherwise.)
3. Find customers with more than 3 orders
4. Get top 3 latest orders for each customer
- 5.Write a query to find customers who placed orders in both January and February 2023.

Department

dept_id	dept_name
10	Engineering
20	HR
30	Finance
40	Marketing

Employees

emp_id	emp_name	dept_id	manager_id	salary
1	Alice	10	NULL	90000.00
2	Bob	10	1	70000.00
3	Charlie	10	1	65000.00
4	David	20	NULL	50000.00
5	Eve	20	4	45000.00
6	Frank	30	NULL	80000.00
7	Grace	30	6	75000.00
8	Hannah	40	NULL	60000.00
9	Ivy	40	8	55000.00
10	Jack	40	8	40000.00

1. Employees per department
2. Manager status classification
3. Departments with average salary > 60000
4. Employees earning above department average
5. Department with highest total salary

Riders

rider_id	rider_name	city
1	Amit	Delhi
2	Rahul	Mumbai
3	Sneha	Pune
4	Priya	Bangalore
5	Karan	Chennai
6	Neha	Delhi
7	Arjun	Hyderabad
8	Riya	Kolkata
9	Varun	Mumbai
10	Meera	Delhi

Rides

ride_id	rider_id	bike_model	kms_driven	ride_date
201	1	Honda Activa	120	2023-01-05
202	2	Royal Enfield	250	2023-01-10
203	3	Bajaj Pulsar	180	2023-02-01
204	1	Honda Activa	90	2023-02-10
205	4	TVS Apache	300	2023-02-15
206	5	Honda Activa	60	2023-02-18
207	2	Royal Enfield	400	2023-03-05
208	6	Yamaha FZ	150	2023-03-10
209	7	Royal Enfield	500	2023-03-12
210	8	TVS Jupiter	100	2023-03-18
211	9	Bajaj Pulsar	220	2023-03-20
212	10	Honda Activa	80	2023-03-22

1. Find total kms driven by each rider
2. Find the rider who has driven the maximum kms
3. Categorize rides by kms (Short < 100, Medium 100–300, Long > 300)
4. Find average kms driven per city
5. Count total rides taken on each bike model

Set:1

1.Find customers who placed at least one order

```
SELECT DISTINCT c.cust_name  
FROM Customers c  
JOIN Orders o ON c.cust_id = o.cust_id;
```

2.Count total orders per customer

```
SELECT c.cust_name, COUNT(o.order_id) AS total_orders  
FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id  
GROUP BY c.cust_name;
```

3.Identify high-value orders("High" if amount > 1000, "Medium" if between 500 and 1000, "Low" otherwise)

```
SELECT order_id,  
       CASE  
         WHEN amount > 1000 THEN 'High'  
         WHEN amount BETWEEN 500 AND 1000 THEN 'Medium'  
         ELSE 'Low'  
       END AS order_category  
FROM Orders;
```

4.Find customers with no orders

```
SELECT c.cust_name  
FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id  
WHERE o.order_id IS NULL;
```

5.Find latest order per customer

```
SELECT c.cust_name, MAX(o.order_date) AS latest_order_date  
FROM Customers c  
JOIN Orders o ON c.cust_id = o.cust_id  
GROUP BY c.cust_name;
```

Set2

1.Total revenue by customer

```
SELECT cust_id, SUM(amount) AS total_revenue
FROM Orders
GROUP BY cust_id;
```

2.Categorize customers based on revenue

```
SELECT cust_id,
CASE
    WHEN SUM(amount) > 5000 THEN 'Gold'
    WHEN SUM(amount) BETWEEN 2000 AND 5000 THEN 'Silver'
    ELSE 'Bronze'
END AS category
FROM Orders
GROUP BY cust_id;
```

3. Find customers with more than 3 orders

```
SELECT cust_id
FROM Orders
GROUP BY cust_id
HAVING COUNT(order_id) > 3;
```

4. Get top 3 latest orders for each customer

```
SELECT cust_id, order_id, order_date, amount
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date DESC) AS rn
    FROM Orders o
) t
WHERE rn <= 3;
```

5. Write a query to find customers who placed orders in both January and February 2023.

```
SELECT cust_id
FROM Orders
WHERE order_date >= '2023-01-01' AND order_date < '2023-03-01'
GROUP BY cust_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 2;
```

Set 3 (Employees + Departments)

1. Employees per department

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY emp_count DESC;
```

2. Manager status classification

```
SELECT emp_name,
       CASE
         WHEN manager_id IS NOT NULL THEN 'Has Manager'
         ELSE 'No Manager'
       END AS manager_status
FROM Employees;
```

3. Departments with average salary > 60000

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

4. Employees earning above department average

```
SELECT e.emp_name, e.salary, e.dept_id
FROM Employees e
JOIN (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM Employees
  GROUP BY dept_id
) d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

5. Department with highest total salary

```
SELECT dept_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY dept_id
ORDER BY total_salary DESC
LIMIT 1;
```

Set4

1. Find total kms driven by each rider

```
SELECT r.rider_name, SUM(b.kms_driven) AS total_kms
FROM Riders r
JOIN Bikes b ON r.rider_id = b.rider_id
GROUP BY r.rider_name;
```

2. Find the rider who has driven the maximum kms

```
SELECT r.rider_name, SUM(b.kms_driven) AS total_kms
FROM Riders r
JOIN Bikes b ON r.rider_id = b.rider_id
GROUP BY r.rider_name
ORDER BY total_kms DESC
LIMIT 1;
```

3. Categorize rides by kms (Short < 100, Medium 100–300, Long > 300)

```
SELECT ride_id, rider_id, kms_driven,
CASE
    WHEN kms_driven < 100 THEN 'Short'
    WHEN kms_driven BETWEEN 100 AND 300 THEN 'Medium'
    ELSE 'Long'
END AS ride_type
FROM Bikes;
```

4. Find average kms driven per city

```
SELECT r.city, AVG(b.kms_driven) AS avg_kms
FROM Riders r
JOIN Bikes b ON r.rider_id = b.rider_id
GROUP BY r.city;
```

5. Count total rides taken on each bike model

```
SELECT b.bike_model, COUNT(b.ride_id) AS total_rides
FROM Bikes b
GROUP BY b.bike_model;
```


Question Set 1 (Simple)

1. What is indexing in SQL, and why is it used?
2. What is the difference between a Primary Key and a Unique Key?
3. What is the difference between DELETE, TRUNCATE, and DROP?
4. What is normalization? Explain its advantages.
5. What is denormalization, and when should it be used?
6. What is a clustered index and a non-clustered index?
7. What is the difference between WHERE and HAVING?
8. What are joins in SQL? Name different types of joins.
9. What is the difference between INNER JOIN and LEFT JOIN?
10. What is a foreign key, and why is it important?
11. What is a view? What are its advantages and limitations?
12. What is a stored procedure?
13. What is the difference between a stored procedure and a function?
14. What is a transaction in SQL?
15. Explain ACID properties in SQL.
16. What is a subquery?
17. What is the difference between a subquery and a join?
18. What is an index scan vs index seek?
19. What is the difference between UNION and UNION ALL?
20. What is a composite key?
21. What are constraints in SQL? Name different types.
22. What is a deadlock?
23. What is the difference between CHAR and VARCHAR?
24. What is the difference between COUNT(*) and COUNT(column_name)?
25. What is a cursor in SQL?
26. What is a trigger?
27. What is the difference between NULL and zero?
28. What is query optimization?
29. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?
30. What is a self join?
31. What is the difference between EXISTS and IN?
32. What are indexes' disadvantages?
33. What is data integrity in SQL?

Answer Set 1 (Simple)

1. Indexing is a database technique used to speed up data retrieval. It creates a data structure that allows faster searching of rows in a table.
2. Primary Key does not allow NULL values and only one per table.
3. Unique Key allows one NULL value and multiple unique keys per table.
4. DELETE removes selected rows, TRUNCATE removes all rows quickly and cannot be rolled back, DROP deletes the entire table structure.
5. Normalization is the process of organizing data to reduce redundancy and improve data integrity.
6. Denormalization is adding redundancy for performance improvement, mainly used in read-heavy systems.
7. Clustered index determines the physical order of data.
8. Non-clustered index stores pointers to data without changing physical order.
9. WHERE filters rows before grouping; HAVING filters groups after aggregation.
10. Joins combine rows from multiple tables. Types include INNER, LEFT, RIGHT, FULL, and CROSS joins.
11. INNER JOIN returns matching records only.
12. LEFT JOIN returns all records from the left table and matched records from the right table.
13. A foreign key enforces relationships between tables and maintains referential integrity.
14. A view is a virtual table created using a SELECT query. It simplifies queries and enhances security but does not store data.
15. A stored procedure is a precompiled SQL statement that can accept parameters and perform operations.
16. Stored procedures can return multiple values; functions return a single value and can be used in SELECT statements.
17. A transaction is a sequence of operations performed as a single logical unit of work.
18. ACID stands for Atomicity, Consistency, Isolation, and Durability—ensuring reliable transactions.
19. A subquery is a query nested inside another query.
20. Subqueries are nested queries; joins combine tables directly and are often faster.
21. Index seek finds specific rows efficiently; index scan scans the entire index.
22. UNION removes duplicates; UNION ALL includes duplicates and is faster.
23. A composite key consists of two or more columns to uniquely identify a row.
24. Constraints enforce rules on data. Types include PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL.
25. A deadlock occurs when two transactions wait for each other to release resources.
26. CHAR has fixed length; VARCHAR has variable length.
27. COUNT(*) counts all rows; COUNT(column_name) counts non-NULL values only.
28. A cursor is used to process rows one at a time.
29. A trigger is a database object that automatically executes in response to events like INSERT or UPDATE.
30. NULL means no value; zero is a numeric value.
31. Query optimization improves query performance by choosing the most efficient execution plan.
32. ROW_NUMBER() assigns unique numbers, RANK() skips numbers on ties, DENSE_RANK() does not skip numbers.
33. A self join is a join where a table is joined with itself.
34. EXISTS checks for existence and stops early; IN compares all values and can be slower.
35. Indexes consume storage space and slow down INSERT, UPDATE, and DELETE operations.

Question Set 2 (Tricky)

1. If a table already has a primary key, do we still need additional indexes?
2. When should you avoid using indexes?
3. Can a table have multiple clustered indexes? Why or why not?
4. What happens internally when you create an index?
5. Why is `SELECT *` discouraged in production queries?
6. How does indexing affect `INSERT` and `UPDATE` performance?
7. Why does `WHERE column != value` not use an index efficiently?
8. What happens if you insert a duplicate value into a unique key?
9. Can foreign keys reference non-primary key columns?
10. Why is `TRUNCATE` faster than `DELETE`?
11. Can views be indexed?
12. Why are `NULL` values problematic in indexing?
13. What causes slow SQL queries in production?
14. How do you identify performance bottlenecks in SQL?
15. Why is `COUNT(1)` sometimes preferred over `COUNT(*)`?

Answers Set 2 (Tricky)

1. Yes, additional indexes may be needed for frequently searched non-key columns.
2. Avoid indexes on frequently updated columns or small tables.
3. No, because only one physical data order is possible.
4. SQL builds a data structure (usually B-tree) to map keys to row locations.
5. It retrieves unnecessary columns and increases I/O.
6. Indexes slow writes because indexes must also be updated.
7. Inequality conditions often cause index scans instead of seeks.
8. The database throws a constraint violation error.
9. Yes, as long as the referenced column has a unique constraint.
10. `TRUNCATE` doesn't log individual row deletions.
11. Yes, indexed views are possible with restrictions.
12. `NULL`s are not stored uniformly and complicate comparisons.
13. Missing indexes, bad joins, large datasets, poor query design.
14. Using execution plans, query profiling, and monitoring tools.
15. Historically optimized in some DBs, though modern engines treat them similarly.

Problem Statement

You are given an employee_logins table that stores employee login activity.

Write an SQL query to **find employee_id(s) who logged in for at least three consecutive days.**

employee_id	employee_name	login_date
101	Alice	2024-01-01
101	Alice	2024-01-02
101	Alice	2024-01-03
101	Alice	2024-01-05
102	Bob	2024-01-01
102	Bob	2024-01-03
102	Bob	2024-01-04
103	Carol	2024-01-10
103	Carol	2024-01-11
103	Carol	2024-01-12
104	David	2024-01-02
104	David	2024-01-04

Requirement

- Consecutive days mean **continuous calendar days**
- Output should contain **only employee_id**
- Order of output does not matter

Expected Result Table

employee_id
101
103

Explanation (for interviewer reference)

- **Employee 101 (Alice)** logged in on
2024-01-01, 2024-01-02, 2024-01-03 → 3 consecutive days
- **Employee 103 (Carol)** logged in on
2024-01-10, 2024-01-11, 2024-01-12 → 3 consecutive days
- Other employees do not have 3 continuous login dates

Problem Statement

You are given a table that stores **family relationships**, where each row represents a person and their father. Write an SQL query to **find the grandfather of a given person**.

person_name	father_name
Amit	Ramesh
Ramesh	Suresh
Suresh	Mahesh
Neha	Ramesh
Rahul	Suresh
Karan	Mahesh
Mahesh	NULL
Pooja	Vinod
Vinod	Anil
Anil	NULL
Sneha	Vinod
Arjun	Anil

Requirement

- Find the **grandfather** of each person
- If grandfather does not exist, exclude the record
- Output should contain:
 - person_name
 - grandfather_name

Explanation

- Amit → Ramesh → Suresh
- Neha → Ramesh → Suresh
- Rahul → Suresh → Mahesh
- Pooja → Vinod → Anil
- Sneha → Vinod → Anil

Expected Result Table

person_name	grandfather_name
Amit	Suresh
Neha	Suresh
Rahul	Mahesh

Problem Statement

You are given **two tables** with identical structures.

Write SQL queries (or determine the result) to find the **number of records returned** when performing the following joins on the **id column**:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- Cross Join

Table A

id	name
1	Alice
2	Bob
3	Carol
4	David
5	Eve
NULL	Unknown_A1
NULL	Unknown_A2
7	Grace
8	Heidi

TABLE B

id	name
1	Adam
2	Brian
4	Diana
NULL	Unknown_B
9	Ivan

Requirement

- Joins must be performed on id
- Determine **number of rows returned**, not the data itself
- Explain how **NULL values affect join results**

```

WITH login_groups AS (
  SELECT
    employee_id,
    login_date,
    DATE_SUB(
      login_date,
      INTERVAL ROW_NUMBER() OVER (
        PARTITION BY employee_id
        ORDER BY login_date
      ) DAY
    ) AS grp
  FROM employee_logins
)
SELECT DISTINCT employee_id
FROM login_groups
GROUP BY employee_id, grp
HAVING COUNT(*) >= 3;

```

```

SELECT
  c.person_name,
  g.father_name AS grandfather_name
FROM family_tree c
JOIN family_tree f
  ON c.father_name = f.person_name
JOIN family_tree g
  ON f.father_name = g.person_name;

```

Question Set 3 (WareHouse, Pyspark, Python and Big Data)

1. What is partitioning and bucketing in a data warehouse, and how do they differ?
2. Explain fact tables and dimension tables. What are their roles in a star schema?
3. What is the difference between OLTP and OLAP systems?
4. What is the difference between an RDD, DataFrame, and Dataset?
5. Explain lazy evaluation in PySpark. Why is it important?
6. What is shuffle in Spark, and why is it expensive?
7. What is the difference between repartition() and coalesce()?
8. Explain narrow vs wide transformations with examples.
9. What is broadcast join in Spark, and when should it be used?
10. How does Spark handle fault tolerance?
11. What is the difference between a list, tuple, and set?
12. What are mutable and immutable data types in Python?
13. Explain list comprehension and its advantages.
14. What is the difference between deep copy and shallow copy?
15. What are `**args` and `kwargs`?
16. What is a generator, and how is it different from a function?
17. Explain Python decorators and their use cases.
18. What is HDFS, and how does it store data?
19. What is the difference between Spark and Hadoop MapReduce?
20. What is data skew, and how do you handle it in big data systems?

Answer Set 3 (WareHouse, Pysarpk, Python and Big Data)

1. What is partitioning and bucketing?

Partitioning divides data into folders based on column values to reduce scanned data.

Bucketing distributes data into fixed files using a hash function to optimize joins.

2. Fact vs Dimension tables

Fact tables store measurable metrics (sales, revenue).

Dimension tables store descriptive attributes (date, product, customer).

3. OLTP vs OLAP

OLTP handles transactional workloads with frequent writes.

OLAP supports analytical queries with complex aggregations.

4. RDD vs DataFrame vs Dataset

RDD is low-level and unstructured.

DataFrame is optimized and schema-based.

Dataset (Scala) adds type safety.

5. Lazy evaluation

Transformations are not executed until an action is triggered, improving optimization.

6. Shuffle

Data redistribution across nodes, expensive due to disk and network I/O.

7. repartition vs coalesce

`repartition()` increases/decreases partitions with shuffle.

`coalesce()` reduces partitions without shuffle.

8. Narrow vs wide transformations

Narrow: data stays in one partition (`map`, `filter`).

Wide: data moves across partitions (`groupBy`, `join`).

9. Broadcast join

Small table is sent to all nodes to avoid shuffle.

10. Fault tolerance

Spark recomputes lost partitions using lineage.

11. List vs Tuple vs Set

List: ordered, mutable.

Tuple: ordered, immutable.

Set: unordered, unique elements.

12. Mutable vs Immutable

Mutable objects can change; immutable cannot.

13. List comprehension

Compact syntax for creating lists, faster and readable.

14. Deep vs Shallow copy

Shallow copy copies references.

Deep copy duplicates all nested objects.

****15. *args and kwargs**

Allow passing variable number of arguments.

16. Generator

Returns items one at a time using yield.

17. Decorators

Functions that modify other functions' behavior.

18. HDFS

Distributed file system storing large data in blocks with replication.

19. Spark vs MapReduce

Spark is in-memory and faster; MapReduce is disk-based.

20. Data skew

Uneven data distribution; handled using salting, repartitioning.

Question 1: String Frequency Counter

Write a Python function that takes a string as input and returns a dictionary containing the **frequency of each character**, excluding spaces and ignoring case.

Example Input:

"Data Engineer"

Expected Output:

{'d': 1, 'a': 2, 't': 1, 'e': 3, 'n': 2, 'g': 1, 'i': 1, 'r': 1}

Question 2: Filter and Map on List of Strings

Given a list of strings, perform the following:

- Filter strings that have **length greater than 4**
- Convert the filtered strings to **uppercase**

Input:

["spark", "sql", "python", "etl", "hadoop"]

Expected Output:

["SPARK", "PYTHON", "HADOOP"]

Question 3: Dictionary Value Transformation

Given a dictionary where values are integers, write a function to:

- Keep only items where the value is **even**
- Square the remaining values

Input:

{"a": 1, "b": 2, "c": 3, "d": 4}

Expected Output:

{"b": 4, "d": 16}

Question 4: Find First Non-Repeating Character

Write a function that returns the **first non-repeating character** in a given string.

Input:

"swiss"

Expected Output:

"w"

Question 5: Group Words by Length Using Dictionary

Given a list of words, group them into a dictionary where:

- The **key** is the length of the word
- The **value** is a list of words of that length

Input:

["data", "ai", "spark", "sql", "python"]

Question 1: Character Frequency Counter

```
def char_frequency(s):  
    freq = {}  
    for ch in s.lower():  
        if ch != " ":  
            freq[ch] = freq.get(ch, 0) + 1  
    return freq
```

Question 2: Filter and Map Strings

```
words = ["spark", "sql", "python", "etl", "hadoop"]
```

```
result = list(  
    map(str.upper,  
        filter(lambda x: len(x) > 4, words))  
)  
print(result)
```

Question 3: Dictionary Value Transformation

```
data = {"a": 1, "b": 2, "c": 3, "d": 4}  
result = {k: v**2 for k, v in data.items() if v % 2 == 0}  
print(result)
```

Question 4: First Non-Repeating Character

```
from collections import Counter
```

```
def first_unique_char(s):  
    count = Counter(s)  
    for ch in s:  
        if count[ch] == 1:  
            return ch
```

Question 5: Group Words by Length

```
def group_by_length(words):  
    result = {}  
    for word in words:  
        result.setdefault(len(word), []).append(word)  
    return result
```